For-Loops in Python

- **Definition**: A for-loop is a sequence of instructions that is repeated for each element in a collection or range.
  - Also called a “definite loop” because it iterates over a predetermined sequence.
- **General Syntax**:

```python
for looping_variable in sequence:
    code block
```

- **Example**: Summing numbers from 1 to 3 using a for-loop

```python
n = 0
for i in range(1, 4):
    n += i
print(n)  # Output: 6
```

- **How it Works**:

1. The range(1, 4) generates a sequence [1, 2, 3].
2. In each iteration, the variable i is updated with the next value from the sequence.
3. The value of n is updated by adding i to it in each iteration.
4. The loop terminates when the sequence ends, resulting in the sum n = 6.

Advanced For-Loops and Applications

- **Iterating over Strings**:
  - Example: Print each character in the string “banana”

```python
for c in "banana":
    print(c)
```

- **Working with Lists**:
  - Example: Summing specific elements from a list using step indexing.

```python
a = [2, 3, 1, 3, 3]
s = 0
for i in range(0, len(a), 2):
    s += a[i]
print(s)  # Output: 6
```

- **Using Dictionaries**:
  - Example: Iterating over keys and values in a dictionary.

```python
dict_a = {"One": 1, "Two": 2, "Three": 3}
for key, value in dict_a.items():
    print(key, value)
```

- **Nested For-Loops**:
  - Example: Summing all elements in a 2D array using nested loops.

```python
x = [[5, 6], [7, 8]]
s = 0
for i in range(len(x)):
    for j in range(len(x[i])):
        s += x[i][j]
print(s)  # Output: 26
```

While Loops in Python

Definition:

- A while loop in Python is a control flow structure that repeatedly executes a block of code as long as the specified condition (logical expression) evaluates to True.

Syntax:

```python
while <condition>:
    # Code block to be repeated
```

Key Concept:

- Before each iteration, Python checks if the condition is true.
- If true, the code block executes and the process repeats.
- If false, the loop ends.

Use Case:

- While loops are particularly useful when the number of iterations is unknown beforehand.

Example of a While Loop

Task:

Determine how many times 8 can be divided by 2 before the result is less than 1.

```python
i = 0
n = 8

while n >= 1:
    n /= 2
    i += 1

print(f'n = {n}, i = {i}')
```

Explanation:

1.i tracks the number of divisions.

2.n starts at 8, and while n >= 1, it is divided by 2.

3.Loop continues until n becomes less than 1.

4.Output: After 4 iterations, n = 0.5 and i = 4.

Infinite Loops and Practical Tips

Infinite Loops:

- If the condition in a while loop always evaluates to True and nothing in the loop changes this condition, the loop runs indefinitely, causing an infinite loop.

Example of Infinite Loop:

```python
n = 0
while n > -1:
    n += 1
```

This loop will never stop as n is always greater than -1.

Prevention:

- Ensure the loop's condition can eventually become False by modifying variables within the loop.
- You can manually interrupt infinite loops by pressing ctrl + c on PC or cmd + c on Mac

Comprehensions in Python

- **Definition**:
  - Comprehensions in Python are a compact and efficient way to create sequences (lists, dictionaries, sets) from other sequences.
  - They provide a more concise syntax compared to traditional loops, which reduces code complexity and improves readability.
- **Common Uses**:
  - List comprehension, dictionary comprehension, and set comprehension.
- **Basic Structure**:

```python
[Output for Input_sequence if Condition]
```

- **Example**:
  - Without comprehension:

```python
x = range(5)
y = []
for i in x:
    y.append(i**2)
print(y)  # [0, 1, 4, 9, 16]
```

  - With comprehension:

```python
y = [i**2 for i in range(5)]
print(y)  # [0, 1, 4, 9, 16]
```

Advanced List Comprehension

- **Conditional List Comprehension**:
  - You can add conditions within the comprehension to filter elements.
  - Example: Only storing even numbers:

```python
y = [i**2 for i in range(5) if i % 2 == 0]
print(y)  # [0, 4, 16]
```

- **Nested Loops in List Comprehension**:
  - Nested loops can also be used within comprehensions to handle more complex logic.
  - Example:

```python
y = [i + j for i in range(5) for j in range(2)]
print(y)  # [0, 1, 1, 2, 2, 3, 3, 4, 4, 5]
```

Dictionary Comprehension

- **Definition**:
  - Similar to list comprehension, but used for creating dictionaries.

- **Basic Structure**:

```python
{key: expression for key, value in iterable}
```

- **Example**:
  - Constructing a dictionary where values are cubed:

```python
x = {'a': 1, 'b': 2, 'c': 3}
new_dict = {key: value**3 for key, value in x.items()}
print(new_dict)  # {'a': 1, 'b': 8, 'c': 27}
```